



Regular Manuscript

# Operator-Constrained Export of Named Entity Recognition for Embedded Linux and Microcontrollers: A Train-Once Graph-Firmware Co-Design with Feasibility Auditing

Submission ID            3e183859-5b07-48d7-b08f-daaaad3e9e2d

Submission Version      Initial Submission

PDF Generation          10 Sep 2026 04:34:03 EST by Atypon ReX

## Authors

|                                                                                                                                                                                                                                                                                                                           |                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Prof. Li Zhao                                                                                                                                                                                                                                                                                                             | <b>Affiliations</b> <ul style="list-style-type: none"><li>• Heilongjiang Jiaotong Polytechnic, Harbin 150025, China</li></ul>     |
| Prof. Ming Zhao                                                                                                                                                                                                                                                                                                           | <b>Affiliations</b> <ul style="list-style-type: none"><li>• Harbin Cambridge University, Harbin 150069, China</li></ul>           |
| Mr. Shu Wang<br><i>Corresponding Author</i><br><i>Submitting Author</i><br><br> <a href="https://orcid.org/0009-0005-3064-1882">ORCID</a><br><a href="https://orcid.org/0009-0005-3064-1882">https://orcid.org/0009-0005-3064-1882</a> | <b>Affiliations</b> <ul style="list-style-type: none"><li>• Heilongjiang Tianyouwei Technology Co., Ltd., Harbin, China</li></ul> |

## Additional Information

**Subject Category**  
Computational and artificial intelligence  
Computers and information processing  
Industrial electronics

**Keywords**  
Edge computing

Embedded systems  
Microcontrollers  
Natural language processing  
**Select a Manuscript Type**  
Research Article

## Files for peer review

All files submitted by the author for peer review are listed below. Files that could not be converted to PDF are indicated; reviewers are able to access them online.

| Name                            | Type of File        | Size     | Page                   |
|---------------------------------|---------------------|----------|------------------------|
| ieee-access-manuscript-flat.pdf | Main Document - PDF | 591.8 KB | <a href="#">Page 4</a> |

# Li Zhao<sup>1</sup>, Ming Zhao<sup>2</sup>, and Shu Wang<sup>3</sup>

<sup>1</sup>Heilongjiang Jiaotong Polytechnic, Harbin 150025, China

<sup>2</sup>Harbin Cambridge University, Harbin 150069, China

<sup>3</sup>Heilongjiang Tianyouwei Technology Co., Ltd., Harbin, China

Shu Wang is the corresponding author (e-mail: shu.wang@hljtyw.com).

Industrial gateways often run named entity recognition (NER) on embedded Linux and on flash-limited microcontrollers (MCUs), but operator sets differ enough that one transformer checkpoint rarely loads on both. Server-class models typically fail with out-of-memory or unsupported-operator errors on MCU runtimes rather than degrading gracefully. EdgeTier-NER is a train-once pipeline built around that constraint: Char-CNN-CRF memory budgeting, ESP-DL-legal quantization, and a fixed graph/firmware split. Neural emissions ship as a 349 KB INT8 ESP-DL graph for the CoNLL export; character embedding and CRF Viterbi stay in firmware; a 698 KB INT16 ONNX graph serves Raspberry Pi adapters. Experiments on CoNLL-2003, CLUENER (66.9 F1 on Raspberry Pi 3B+ and Pi 400 INT16 ONNX, 66.7 F1 on ESP32-S3 INT8), a latency-only sequence-length sweep ( $L \in \{16, 32, 64, 96, 128\}$ ), Pi 400 portability, and a graph-cut ablation evaluate deployability without conflating placement with duty-cycle policies. The audit covers one export family: OOM and unsupported-operator boundaries, F1, latency, and peak memory. On ESP32-S3, firmware embedding and quantization take about 64% of end-to-end MCU time and grow with sentence length, while ESP-DL inference stays nearly flat. That split points to variable-length data movement outside the accelerator graph as the main NLP bottleneck on MCU.

Edge computing, embedded systems, graph-firmware co-design, model compression, named entity recognition, operator constraints, TinyML

Industrial IoT gateways, in-vehicle units, and field terminals often need on-device language understanding when cloud links are slow or unavailable [1], [2]. Named entity recognition (NER) tags people, locations, organizations, and domain terms in short utterances, log lines, and maintenance reports. Edge SKUs still cap flash, SRAM/PSRAM, supported neural operators, and end-to-end latency; one product line may ship both a Linux SBC and an MCU.

The systems problem here is operator-set heterogeneity under one trained checkpoint. An ESP32-S3 module has about 512 KB internal SRAM and flash-backed model stor-

age; a Raspberry Pi-class board has gigabyte-scale RAM, roughly three orders of magnitude more headroom than a typical transformer checkpoint [3]. MCU runtimes such as ESP-DL reject LayerNorm and multi-head attention, so unsupported-operator failures matter as much as out-of-memory (OOM) on these boards [1], [4], [5]. A usable pipeline must co-design the encoder, export graph, and firmware decode path so the export runs on the board; leaderboard F1 alone is not enough.

Cross-ISA edge vision work often studies latency surrogates and adaptive subnet selection under iso-FLOPs budgets. Sequence labeling on MCUs adds a different

set of constraints: runtime operator whitelists push part of the model into firmware, variable-length token loops consume wall time outside the neural accelerator, and linear-chain CRF decoders usually stay in firmware. The focus is feasibility auditing and stage attribution, not tier-selection policies or duty-cycle crossover bounds.

Most NER papers stop before board-level export. Label-efficient methods based on prototypical networks and structured nearest neighbors [6]–[10] assume GPU-class inference. Compact sequence labelers for embedded Linux [11], [12] rarely publish end-to-end MCU numbers with operator-feasibility tables. Recent ESP32 language-model demos focus on generation throughput [13]; prior work does not pair Linux and MCU NER audits with OOM/Unsup. rows for transformer baselines under one export family.

EdgeTier-NER is a train-once pipeline built for board-level export. It trades some CoNLL F1 against BiLSTM or BERT for a flash-legal INT8 graph, a documented memory map, tiered checkpoint auditing, and reproducible UART evaluation. The architecture rests on character-level encoding for memory budgeting, quantization and residual rewrites under ESP-DL operator rules, and stage-level profiling that separates firmware embed work from accelerator inference.

Our contributions are:

- **C1.** An operator-feasibility audit protocol: a matrix of OOM, unsupported operators, and PASS outcomes for transformer and compact NER baselines on Pi and ESP32-S3 under one checkpoint family, with runtime legality reported next to F1 (Table 7; Table 1).
- **C2.** Graph/firmware co-design under the ESP-DL whitelist: post-embedding graph cut, identity  $1 \times 1$  skip convolutions in place of illegal residual Adds, and firmware-side character embedding plus CRF Viterbi, yielding 698 KB INT16 ONNX (Pi) and 349 KB INT8 .espd1 (ESP32) with a documented memory map (Sections III-D, IV).
- **C3.** Sequence-length-aware stage attribution: firmware timers show embed+quant at about 64% of MCU wall time and scaling with sentence length, while ESP-DL infer stays nearly flat; most of the MCU second is spent outside the accelerator graph on variable-length token work (Sections V-C, V-E).

Our scope is single-checkpoint export feasibility after training: tier placement, duty-cycle scheduling, and cross-ISA dynamic inference are out of scope.

Sections II–VI cover related work, methodology, deployment architecture, experiments, and conclusions.

## A. SEQUENCE LABELING AND CLASSICAL NER

Early neural NER systems combine word representations with recurrent encoders and linear-chain conditional random fields (CRFs) to model tag dependencies [11], [12], [14], [15]. CRFs remain attractive for edge deployment because

Viterbi decoding is a lightweight dynamic program that can run in firmware without neural acceleration [16]. Character-aware encoders further reduce vocabulary pressure for morphologically rich or domain-specific text [12]. Our work builds on this line but replaces LSTM stacks with dilated convolutions and explicitly sizes the encoder for MCU export rather than server-side accuracy alone.

## B. FEW-SHOT AND LABEL-EFFICIENT NER

Episodic few-shot NER builds on model-agnostic meta-learning [17], prototypical and matching networks [6], [18], early task-specific few-shot NER [19], structured nearest-neighbor decoding [7], [20], and dedicated benchmarks [8]–[10]. Those systems are evaluated almost exclusively on GPU servers and do not analyze MCU flash/SRAM fit or operator legality; we use episodic training only as an optional stress test of the same compact encoder.

## C. MODEL COMPRESSION FOR NLP

Knowledge distillation [21] transfers soft label structure from a teacher to a smaller student; DistilBERT [22], TinyBERT [23], and MobileBERT [24] apply this paradigm to transformers for mobile and edge servers. Post-training quantization [25], [26] reduces weight and activation bit-width; however, transformer graphs rely on operators that are absent or inefficient on MCU runtimes. Pruning and Huffman coding further shrink models but rarely bring BERT-class networks below single-digit megabytes. Our compression pipeline picks the student architecture for the target hardware: Conv1d, BatchNorm, ReLU, Gemm, and Add, all with known ESP-DL support [4]. Distillation aligns per-token emissions and CRF transition matrices so that INT8/INT16 quantization preserves boundary decisions.

## D. EDGE AND TINYML FOR NLP

TinyML frameworks such as TensorFlow Lite for Microcontrollers [2] and the TinyML textbook ecosystem [1] have mature tooling for keyword spotting and small vision models, but NLP on MCUs remains sparse. MCU DNN deployment work covers MCUNet’s search for tiny ImageNet models [27], DORY’s automated DNN-to-MCU memory/scheduling stack [28], and surveys of machine-learning offloading in multi-access edge computing [29]. These lines emphasize flash/SRAM co-design and measured latency/energy more than server-class F1 alone. Contextual string embeddings (Flair) [30] improve GPU-side sequence labeling but depend on large character-LM encoders. ESP-DL [4] enables int8/int16 inference on ESP32-S3 under a strict operator set; models with LayerNorm or attention are typically blocked. Small-scale LLMs (0.5B–2B parameters; e.g., Qwen3-0.6B at about 330 MB INT4 weights [31]; MiniCPM-2B at about 1.2 GB INT4 [32]) remain orders of magnitude above 8 MB flash (Table 7), so they are outside our C1 feasibility audit for on-device NER. MobileNet-style depthwise designs [5] inspire our use of efficient

Comparison with prior edge and TinyML deployment work. ✓ = reported; — = not addressed; (partial) = limited or indirect. **Ours (CoNLL)**: 349 KB ESP32 / 698 KB Pi ONNX; 69.5 F1 @ 1.7 s (MCU), 69.1 F1 @ 54.7 ms (Pi); *espd1\_test* PASS; OOM/Unsup. for transformers in Table 7. **Ours (CLUENER)**: 351 KB ESP32; 66.7 F1 @ 3.5 s (MCU,  $n=1,343$ ), 66.9 F1 @ ~69 ms (Pi); ≤0.2 pt drop vs Pi; same density + identity-skip export.

| Work                  | Task       | MCU | Linux | Size      | Lat./energy | OOM/Unsup. |
|-----------------------|------------|-----|-------|-----------|-------------|------------|
| MCUNet [27]           | Vision     | ✓   | —     | (partial) | ✓           | —          |
| DORY [28]             | Vision DNN | ✓   | —     | ✓         | ✓           | —          |
| TFLM [2]              | KWS/vision | ✓   | —     | ✓         | (partial)   | —          |
| DistilBERT [22]       | NLP        | —   | ✓     | ✓         | (partial)   | (partial)  |
| ESP-DL [4]            | Runtime    | ✓   | —     | —         | —           | (partial)  |
| esp32-ai TLMs         | LM gen.    | ✓   | —     | ✓         | ✓           | —          |
| Flair [30]            | NER        | —   | ✓     | —         | —           | —          |
| <b>Ours (CoNLL)</b>   | NER        | ✓   | ✓     | ✓         | ✓           | ✓          |
| <b>Ours (CLUENER)</b> | NER        | ✓   | ✓     | ✓         | ✓           | ✓          |

Positioning against prior work. ✓ = addressed; — = not addressed; (partial) = limited coverage.

| Capability                     | Few-shot NER | NLP compr. | Edge NER  | Ours      |
|--------------------------------|--------------|------------|-----------|-----------|
| MCU (ESP32-class) deployment   | —            | —          | —         | ✓         |
| Unified train-once pipeline    | —            | —          | —         | ✓         |
| Reported latency + peak memory | —            | (partial)  | (partial) | ✓         |
| Sub-MB model size              | —            | (partial)  | (partial) | ✓         |
| Linux SBC deployment           | ✓            | ✓          | ✓         | ✓         |
| Few-shot / low-label training  | ✓            | —          | —         | (partial) |

convolutions rather than recurrence or attention at the MCU tier.

## E. POSITIONING OF THIS WORK

Table 1 compares EdgeTier-NER with representative edge and TinyML deployment lines on task coverage, MCU/Linux support, reported size/latency, and whether OOM or unsupported-operator outcomes are documented. Table 2 summarizes capability coverage at a higher level.

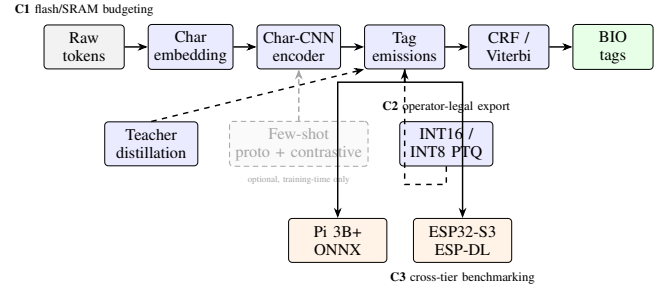
Few label-efficient NER papers report whether the encoder fits MCU flash and operator whitelists; we keep OOM and Unsup. rows in the tables rather than omitting infeasible baselines. Many edge NER systems stop at embedded Linux; we also run the same checkpoint on ESP32-S3 (about 1.72 s/sentence, 349 KB flash graph, stage timers, on-device self-test). Vision-oriented MCU stacks assume fixed-resolution graphs; NER adds variable-length firmware embed loops and external CRF decode.

## A. PROBLEM FORMULATION

Given an input sentence tokenized into  $\mathbf{x} = (x_1, \dots, x_T)$ , NER assigns a tag  $y_t \in \mathcal{Y}$  to each token [11], [16]. At deployment time the system must map a sentence to a tag sequence under architectural constraints

$$\begin{aligned} |\theta|_{\text{flash}} &\leq M_{\text{flash}}, & |\theta|_{\text{act}} &\leq M_{\text{act}}, \\ \tau(x) &\leq \tau_{\text{max}}, & \text{Ops}(\theta) &\subseteq \mathcal{O}_{\text{runtime}}, \end{aligned} \quad (1)$$

where  $|\theta|_{\text{flash}}$  is serialized model storage,  $|\theta|_{\text{act}}$  is the peak activation buffer,  $\tau(x)$  is end-to-end latency, and  $\mathcal{O}_{\text{runtime}}$  is the operator set of the target runtime (ONNX Runtime on Pi; ESP-DL on ESP32-S3). A few-shot training mode with episodic  $N$ -way  $K$ -shot tasks [6], [8] is optional; annotation-efficiency results appear only in Section V-D.



Training and compression overview (C1–C3). Few-shot branch is dashed (training only); deployment adapters in Section IV.

## B. OVERALL FRAMEWORK

Figure 1 summarizes EdgeTier-NER. Training proceeds in three stages: (1) full-data (or optional few-shot) learning of the Char-CNN-CRF encoder; (2) optional distillation from a stronger teacher; (3) post-training quantization to INT16 (Pi) or INT8 (MCU) with ESP-DL operator validation. Export splits the graph into a neural emissions block and a firmware decoding block (Section IV).

## C. FLASH/SRAM-AWARE ENCODER BUDGETING (C1)

Word-level NER allocates an embedding matrix  $\mathbf{E} \in \mathbb{R}^{|V| \times d}$  that dominates flash when  $|V| > 10^4$ . Character-level encoders compose a much smaller table  $\mathbf{E}_c \in \mathbb{R}^{|V_c| \times d}$  with shared convolutions [12], [14], reducing vocabulary-dependent storage from  $O(|V|d)$  to  $O(|V_c|d)$  with  $|V_c| \ll |V|$ . For the CoNLL-2003 export,  $|V_c| = 60$  and  $d = 64$  yield a character table under 16 KB FP32, compared with about 8 MB for a representative word BiLSTM embedding. C1 is a memory-budgeting decision on the MCU flash hierarchy, not an accuracy-seeking model choice.

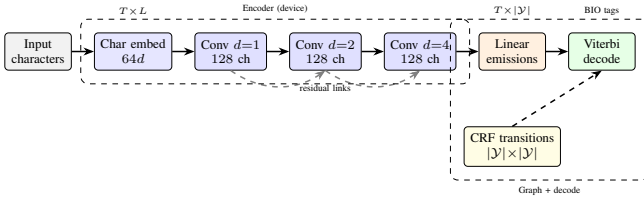
Script choice changes the budget. Chinese CLUENER [33] induces  $|V_c|=3,673$  after pruning (918 KB FP32 embedding; Table 3). Byte-level tokenization ( $|V|=258$ ) restores a 64.5 KB embedding plus 754 KB ONNX graph at a measured cost of about 5.7 F1 points on full-data CLUENER, illustrating the same storage-versus-accuracy lever for logographic text.

Each token is padded or truncated to  $W=20$  characters. Conv1d and max-over-characters pooling produce width-128 token vectors; three dilated Conv1d blocks (kernel 3, dilations (1, 2, 4)) with BatchNorm, ReLU, and residuals expand context while staying inside ESP-DL-supported operators [4], [5]. A linear layer emits scores  $\mathbf{E}_t \in \mathbb{R}^{|\mathcal{Y}|}$ ; a linear-chain CRF with transitions  $\mathbf{T}$  defines

$$p(y | x) = \frac{\exp(\sum_t \mathbf{E}_t[y_t] + \mathbf{T}[y_{t-1}, y_t])}{Z(x)}. \quad (2)$$

Figure 2 details the encoder–CRF stack. Table 3 quantifies dominant memory terms.





Char-CNN-CRF encoder and CRF Viterbi. Embed/decode in firmware; quantized emissions graph in flash.

Measured memory budget by language/script (FP32 embedding table + serialized emissions graph). CoNLL/CLUENER from export measurements; word-level BiLSTM is host-only (not MCU-exportable).

| Component                | Word BiLSTM  | Char (CoNLL)              | Char (CLUENER)             |
|--------------------------|--------------|---------------------------|----------------------------|
| Embedding table          | ~8.0 MB      | 15 KB ( $ V_c =60$ )      | 918 KB ( $ V_c =3,673$ )   |
| Encoder + graph          | ~9.1 MB ckpt | 698 KB ONNX / 349 KB INT8 | 1.61 MB ONNX               |
| Peak act. (128 tok.)     | ~1-2 MB      | ~160 KB                   | ~128 KB                    |
| Byte-level ( $ V =258$ ) | —            | —                         | 64.5 KB emb. + 754 KB ONNX |

## D. OPERATOR-LEGAL COMPRESSION AND EXPORT (C2)

When a teacher is available we minimize [21]

$$\mathcal{L}_{\text{distill}} = \alpha T^2 \text{KL} \left( \sigma \left( \frac{\mathbf{E}^S}{T} \right) \parallel \sigma \left( \frac{\mathbf{E}^T}{T} \right) \right) + \beta \|\mathbf{T}^S - \mathbf{T}^T\|_2^2, \quad (3)$$

aligning both student/teacher emissions and CRF transitions ( $T=2.0$ ,  $\alpha=0.5$ ,  $\beta=0.1$  for the reported BERT-teacher run). Transition matching matters for boundary decisions after INT8 quantization.

Post-training quantization follows integer-only practice [25]: calibration on the development split, per-channel weight scales, INT16 ONNX for Pi and INT8 .espdl for ESP-DL. Three export choices matter for MCU legality. BatchNorm is folded into preceding convolutions to reduce runtime operator count. The post-embedding graph cut runs embedding lookup in firmware and starts the .espdl graph after embedding, which keeps the largest mutable activation buffer in octal PSRAM rather than internal SRAM. Identity  $1 \times 1$  skip convolutions replace residual skip-from-input Add so residual paths match ESP-DL tensor layouts while preserving skip connectivity (Section V-B). Few-shot objectives (prototypical + contrastive [6], [34]) remain available at training time; they do not change the exported operator set.

## E. FOUR-TIER EXPORT AUDIT LADDER

Operator-feasibility auditing must localize accuracy loss to training, export, or on-device execution rather than reporting a single board F1. Before sign-off we run a reusable four-tier ladder (scripts/cluener\_p0\_tier\_ladder.py): **L1** PyTorch FP32 on the checkpoint; **L2** INT16 ONNX deploy bundle; **L3** PPQ INT8 host simulation of the ESP-DL graph; **L4** UART inference on ESP32-S3. L1–L3 share the same IOB2 entity-level metric on an identical 200-sentence prefix and report mean emission cosine vs. L1; L4 reports the full-split UART result, so its  $\Delta$  is not directly

Four-tier export audit on CLUENER (Gate G2). L1–L3: 200-sentence prefix; L4: full UART test ( $n=1,343$ ). Emission cosine vs. L1.

| Tier                      | F1   | $\Delta$ vs L1 | Emission cos. |
|---------------------------|------|----------------|---------------|
| L1 PyTorch FP32           | 69.1 | —              | —             |
| L2 INT16 ONNX             | 68.5 | −0.6           | 0.998         |
| L3 PPQ INT8 (host)        | 67.3 | −1.8           | 0.993         |
| L4 ESP32 UART (full test) | 66.7 | −†             | —             |

†L4 uses the full CLUENER test split ( $n=1,343$ );  $\Delta$  vs L1 is not comparable across splits. Gate G2 board F1: 66.9 Pi / 66.7 ESP32 (Table 11, Config A).

comparable (Table 4). If L1 is healthy but a later tier drops with low cosine, the fault lies in export, quantization, or firmware encode; if all tiers track L1 within a narrow band, the export chain preserved checkpoint semantics.

Embedded boot-time espdl\_test validates only that the flashed graph loads and that golden word\_features → emissions tensors match within tolerance. It does not bound entity-level F1; our audits therefore treat self-test as necessary but insufficient and require L1 host F1 plus tier parity for deployment sign-off.

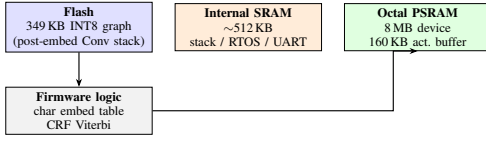
Table 4 shows the CLUENER Gate G2 ladder for checkpoint ckpt-cluener-full-s42 (bundle edgefs-esp32-08549f7c): on a shared 200-sentence prefix, L2 remains within 0.6 F1 points of L1 with emission cosine 0.998, while L3 INT8 simulation accounts for most of the remaining gap; L4 UART on the full test split ( $n=1,343$ ) reaches 66.7 F1 (cluener\_p0\_tier\_ladder.json).

This section specifies how a single trained checkpoint is placed on each board: memory regions, graph cut points, and adapters as an auditable systems design, not a server-only accuracy pipeline.

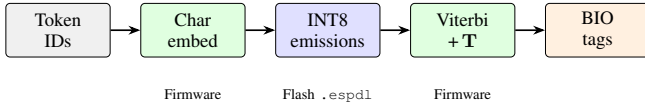
### A. MEMORY MAP ON ESP32-S3

Figure 3 shows the working placement for the pure INT8 path. Quantized weights sit in flash as a 349 KB post-embedding .espdl graph. Internal SRAM (about 512 KB) retains FreeRTOS stacks, UART logging, and control state. The largest mutable activation buffer (160 KB for the 128-token emissions path) is allocated from 8 MB octal PSRAM so that peak tensor traffic does not contend with RTOS critical sections in internal SRAM. Character embedding tables remain in firmware: they are small after CoNLL pruning ( $|V_c|=60$ ) and must feed a graph whose first tensor is already embedded. The export cut sits after embedding for that reason (Section III-D).

Pi 3B+ uses a different hierarchy (process RSS): the 698 KB INT16 ONNX emissions graph runs under ONNX Runtime, with CRF transitions and Viterbi in NumPy; peak RSS of 62.2 MB includes the interpreter and Python heap and is not comparable byte-for-byte to MCU SRAM, but is the relevant Linux-SBC budget metric.



ESP32-S3 memory map (INT8): flash .espdl, firmware embed/Viterbi, PSRAM activations, internal SRAM for RTOS/UART.



Graph/firmware split: character embedding and CRF Viterbi remain outside the .espdl/ONNX emissions graph.

## B. GRAPH/FIRMWARE SPLIT

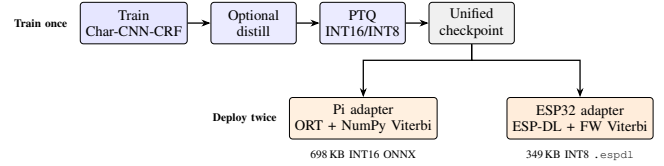
Figure 4 summarizes the cut between the neural emissions block and the decoding block. Conv/linear emissions stay in the quantized runtime graph because they dominate FLOPs and benefit from ESP-DL kernels. CRF Viterbi is a small dynamic program over  $|\mathcal{Y}| \times |\mathcal{Y}|$  transitions; implementing it in firmware (or NumPy on Pi) avoids unsupported control-flow operators inside the neural runtime and keeps transition tables easy to update after distillation. The cut is both an operator-compliance decision and a buffer-lifecycle decision: the graph owns dense activations; firmware owns discrete decode state.

## C. TRAIN-ONCE / DEPLOY-TWICE ADAPTERS

Figure 5 shows the adapter view. A single training run produces a checkpoint family; bit-width and packaging differ by tier, but emissions semantics are shared. On Pi, the adapter loads INT16 ONNX emissions, runs ORT, decodes with exported `crf_transitions`, and reports mean latency over warmup 3 / repeat 10. On ESP32, the adapter loads INT8 .espdl from flash, runs firmware embedding, ESP-DL inference, and firmware Viterbi; a UART harness streams the test set and `espdl_test` validates the embedded golden path before full evaluation. Feasibility for baselines is reported as OOM or Unsup. when packaging or kernels fail; omitting those rows would hide the architectural boundary that motivates the compact export.

## A. PLATFORMS, DATASETS, AND PROTOCOL

Hardware includes Raspberry Pi 3 Model B Plus (BCM2837B0, Cortex-A53 @ 1.4 GHz, 1 GB RAM), Raspberry Pi 400 (BCM2711, Cortex-A72 @ 1.8 GHz, 4 GB RAM; Gate G3), and ESP32-S3-DevKitC-1 (240 MHz, 512 KB internal SRAM, 8 MB flash, 8 MB octal PSRAM). Both Linux boards run the same INT16 ONNX export; Pi 400 checks cross-SKU portability without changing graph



Train-once / deploy-twice: unified checkpoint consumed by Pi ORT and ESP32 ESP-DL adapters.

semantics. Training uses an NVIDIA RTX 5090 workstation (PyTorch).

Datasets are CoNLL-2003 [35], OntoNotes 5.0 [36], and CLUENER [33] (Table 5), converted to BIO with a 128-token edge inference cap. Main accuracy uses official `CoNLL eng.testb` ( $n=3,453$ ); hardware tables use a truncated aligned prefix ( $n=1,722$  on Pi,  $n=1,721$  on ESP32) for Pi/ESP32 parity. The one-sentence gap on ESP32 reflects UART retry exhaustion on a single long sentence, not a protocol mismatch.

Baselines include BiLSTM-CRF, BERT-base-CRF, DistilBERT-CRF, and TinyBERT [3], [11], [22], [23]; few-shot ProtoBERT / NNShot / StructShot appear only as a compact annotation-efficiency check. Methods that exceed memory or require unsupported operators are marked **OOM** or **Unsup.** [1]. Reported metrics are serialized size, mean end-to-end latency, peak memory (RSS on Pi; SRAM/PSRAM on ESP32), and `espdl_test` for the MCU graph. Pi benchmarks use warmup 3, repeat 10 (Table 6).

Dataset statistics. Sentence counts follow the standard splits used in prior NER benchmarks [33], [35], [36].

| Dataset       | Lang. | Train  | Dev   | Test  | # Types | Tag |
|---------------|-------|--------|-------|-------|---------|-----|
| CoNLL-2003    | EN    | 14,041 | 3,250 | 3,453 | 4       | BIO |
| OntoNotes 5.0 | EN    | 59,924 | 8,528 | 8,262 | 18      | BIO |
| CLUENER       | ZH    | 10,748 | 1,343 | 1,345 | 10      | BIO |

Key hyperparameters and deployment settings.

| Setting                                      | Value                         |
|----------------------------------------------|-------------------------------|
| Optimizer / learning rate                    | AdamW, $1 \times 10^{-3}$     |
| Batch size / max sequence length             | 32 / 128 tokens               |
| Char-CNN: embed dim / filters / dilations    | 64 / 128 / (1, 2, 4)          |
| Dropout                                      | 0.1                           |
| Few-shot episodes (CoNLL-2003)               | 4-way 1-shot and 4-way 5-shot |
| $\lambda$ (contrastive), $\tau_p$ , $\tau_c$ | 0.1, 10, 0.1                  |
| Distill $\alpha$ , temperature $T$ , $\beta$ | 0.5, 2.0, 0.1                 |
| Pi export                                    | INT16 ONNX + Python Viterbi   |
| ESP32 export                                 | INT8 .espdl + firmware embed  |
| Pi benchmark                                 | warmup 3, repeat 10           |



Deployment feasibility on Pi 3B+ and ESP32-S3-DevKitC-1 (8 MB flash, 8 MB PSRAM, 512 KB SRAM). OOM = out of memory; Unsup. = unsupported operators. Latency/RSS measured for Ours only. Pi INT16: 69.8 KB ONNX, 69.1 F1 @ 54.7 ms ( $n=1,722$ ). ESP32 INT8 (densified dilations + identity-skip residuals): 1,719 ms / 69.5 F1 ( $n=1,721$ , ESP-DL self-test PASS). Transformer and small-LLM rows are not timed on Pi: their multi-hundred-MB to multi-GB footprints exceed the sub-MB flash budget that defines our product SKU, even if peak RSS might fit 1 GB RAM. Hybrid fall-back: Section V-B.

| Method            | Size    | Pi Lat. | Pi Mem. | ESP32 Lat. | ESP32 Mem. |
|-------------------|---------|---------|---------|------------|------------|
| BERT-base-CRF     | ~400 MB | —       | —       | OOM        | OOM        |
| DistilBERT        | ~250 MB | —       | —       | OOM/Unsup. | —          |
| ProtoBERT         | ~400 MB | —       | —       | OOM        | OOM        |
| Qwen3-0.6B (INT4) | ~330 MB | —       | —       | OOM        | OOM        |
| MiniCPM-2B (INT4) | ~1.2 GB | —       | —       | OOM        | OOM        |
| Ours (INT16)      | 698 KB  | 54.7 ms | 62.2 MB | —          | —          |
| Ours (INT8)       | 349 KB  | —       | —       | 1,719 ms   | 160 KB     |

## B. DEPLOYMENT FEASIBILITY AND EFFICIENCY

Table 7 records whether each method runs on Pi and ESP32. BERT-base-CRF and ProtoBERT OOM on ESP32-S3 even with PSRAM; DistilBERT also hits **Unsup.** (LayerNorm / attention kernels missing from ESP-DL).

On Pi 3B+, Ours INT16 (698 KB ONNX) reaches **69.1 F1 @ 54.7 ms** mean latency and **62.2 MB** peak RSS on the truncated aligned split ( $n=1,722$ ). The same export on Raspberry Pi 400 (Gate G3; BCM2711, 4 GB RAM) yields **69.1 F1 @ 37.6 ms** and **69.0 MB** peak RSS on the same split, with no graph changes. Host ONNX on the same export matches 69.1 F1 at 3.2 ms/sentence.

On ESP32-S3 the flash-budget path is a single pure ESP-DL INT8 configuration with on-device self-test PASS. The export uses a post-pooling layout and identity  $1 \times 1$  skip convolutions in place of residual skip-from-input Add (hereafter the identity-skip residual fix; densified dilations keep the same legal graph). This is an export-time layout change, not a training hyperparameter. UART evaluation ( $n=1,721$ ) reports **69.5 F1** (P75.9 / R64.0) at **1,719 ms** mean (about 1.72 s; p50 1,298 ms, p95 3,484 ms), within 0.4 points of Pi/Host (69.1) and 0.3 points of host PPQ INT8 (69.8). Sub-point INT8-vs-INT16 ordering (ESP32 69.5 vs. Pi 69.1; host PPQ 69.8 vs. host ONNX 69.1) lies within post-training quantization calibration noise and should not be read as lower bit-width outperforming INT16. The .espdll is 349 KB; densified dilated kernels and identity-skip  $1 \times 1$  convolutions trade latency for ESP-DL-legal residuals. An optional hybrid fall-back (INT8 encoder + FP32 blocks) reaches 69.3 F1 @ about 3.70 s and is not the Table 7 latency cell. Pure-INT8 throughput is about 2,100 sentences/hour versus about 66,000 on Pi. DistilBERT and TinyBERT are not timed on Pi: their checkpoints are two orders of magnitude above the sub-MB flash budget that defines the product claim, so a RAM-only Pi run would not answer the deployment question posed here.

## C. STAGE ATTRIBUTION

Stage-latency decomposition (ESP32 pure INT8,  $n=1,721$ ; esp\_timer). Pi e2e from Table 7; Pi stages not instrumented.

| Path       | End-to-end | Embed+quant | ESP-DL infer | CRF decode |
|------------|------------|-------------|--------------|------------|
| ESP32 INT8 | 1,719 ms   | 1,104 ms    | 615 ms       | 0.23 ms    |
| Pi INT16   | 54.7 ms    | —           | —            | —          |

End-to-end MCU latency alone does not show where time is spent on variable-length NER inputs. Stage timers on the same pure-INT8 path used for Table 7 ( $n=1,721$ ; on-device F1 69.5 with ESP-DL self-test PASS) split end-to-end time as follows. Mean 1,719 ms end-to-end breaks down into embed+quant 1,104 ms (64%), ESP-DL infer 615 ms (36%), and CRF Viterbi 0.23 ms ( $<0.1\%$ ) per esp\_timer. Embed+quant (firmware char-CNN feature pack) dominates; ESP-DL cost is nearly constant across sentence lengths in the CoNLL-aligned prefix, while Viterbi is negligible. Densified ESP-DL kernels are not the sole MCU bottleneck: firmware embedding still accounts for nearly two-thirds of the 1.72 s path.

## D. ACCURACY CONTEXT AND ANNOTATION-EFFICIENCY NOTE

Table 9 situates the compact export against server-class F1 on official eng.testb: Ours 71.4 $\pm$ 0.9 ( $k=3$ ), BiLSTM 73.4 $\pm$ 0.4 ( $k=3$ ), BERT 85.8 $\pm$ 0.4 ( $k=3$ ), DistilBERT 86.6 $\pm$ 0.1 ( $k=3$ ), TinyBERT 84.4 ( $k=1$ ).

Main results (entity-level IOB2 F1). CoNLL-2003: official eng.testb ( $n=3,453$ ); Ours/BiLSTM/BERT/DistilBERT: mean  $\pm$  std,  $k=3$ ; TinyBERT:  $k=1$ . OntoNotes/CLUENER baselines: single edge-protocol runs.

| Method                        | CoNLL-2003     | OntoNotes | CLUENER |
|-------------------------------|----------------|-----------|---------|
| BiLSTM-CRF <sup>†</sup>       | 73.4 $\pm$ 0.4 | 75.7      | 67.0    |
| BERT-base-CRF <sup>†</sup>    | 85.8 $\pm$ 0.4 | 83.8      | 75.2    |
| DistilBERT-CRF <sup>†,‡</sup> | 86.6 $\pm$ 0.1 | 83.1      | 71.9    |
| TinyBERT <sup>†,‡</sup>       | 84.4           | 72.3      | 71.0    |
| Ours                          | 71.4 $\pm$ 0.9 | 73.8      | 67.6    |

<sup>†</sup>BiLSTM-CRF: word-level GloVe baseline (configs/model/bilstm\_crf\_glove.yaml; lowercase, no char encoder). All baselines re-trained under the same edge protocol as Ours (BIO, 128-token cap; Table 6), not tuned to published CoNLL peaks ( $\sim 88$ –91 BiLSTM /  $\sim 91$  BERT).

<sup>‡</sup>DistilBERT exceeds BERT-base by 0.8 F1 on CoNLL (86.6 $\pm$ 0.1 vs. 85.8 $\pm$ 0.4;  $k=3$ ); TinyBERT drops more on OntoNotes.

The BiLSTM-CRF row is a word-level baseline (GloVe 6B 100d, fine-tuned; configs/model/bilstm\_crf\_glove.yaml) with lowercase:true preprocessing and no character encoder, unlike the BLSTM-CNN-CRF lines in [12], [15]. Lowercasing removes case cues that English NER exploits, and the absence of a char branch blocks recovery of surface-form information; these protocol choices explain most of the gap to commonly cited 88–91 sentence-level CNN-augmented BiLSTM results. BiLSTM uses a uniform  $k=3$  training recipe (batch 256, 30 epochs; per-seed F1 73.6/73.8/72.9), independent of the transformer baseline runs. Baselines follow our edge protocol (BIO labels, 128-token cap; Table 6) without literature-matching hyperparameter search; within-table gaps are the intended comparison. On CoNLL, DistilBERT mean exceeds BERT mean by 0.8 F1 (about two standard deviations of the BERT seed spread;  $k=3$ ), while TinyBERT falls further on

OntoNotes; that gap is not tested for significance. OntoNotes / CLUENER Ours are 73.8 / 67.6 (FP32 host training; CLUENER deploy F1 is 66.9/66.7 in Gate G2). The gap to BiLSTM/BERT is the cost of a flash-legal graph when board export matters more than peak F1.

Table 10 briefly records 4-way 1/5-shot F1.

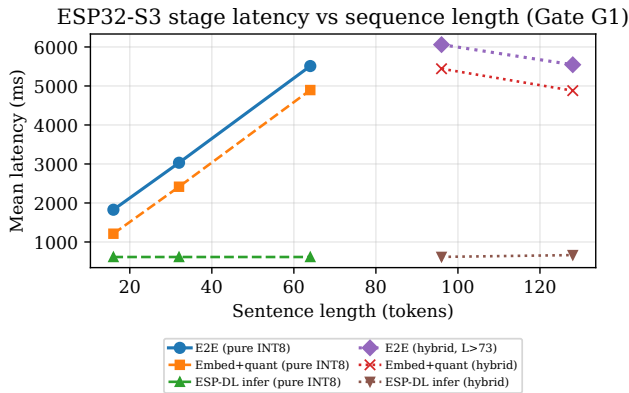
Few-shot results (entity-level F1 mean  $\pm$  std over five seeds). All episodes use 4-way 1-shot / 4-way 5-shot protocols.

| Method               | 4-way 1-shot    | 4-way 5-shot    |
|----------------------|-----------------|-----------------|
| <i>CoNLL-2003</i>    |                 |                 |
| ProtoBERT            | 58.8 $\pm$ 2.1  | 61.3 $\pm$ 1.6  |
| Frozen-BERT Proto    | 4.1 $\pm$ 0.6   | 4.3 $\pm$ 0.5   |
| NNShot               | 10.8 $\pm$ 2.4  | 20.6 $\pm$ 1.2  |
| StructShot           | 59.1 $\pm$ 1.0  | 61.0 $\pm$ 1.9  |
| <b>Ours</b>          | 29.3 $\pm$ 2.0  | 30.9 $\pm$ 2.3  |
| <i>OntoNotes 5.0</i> |                 |                 |
| ProtoBERT            | 23.7 $\pm$ 18.9 | 36.1 $\pm$ 12.5 |
| NNShot               | 4.1 $\pm$ 2.6   | 8.6 $\pm$ 1.0   |
| StructShot           | 18.8 $\pm$ 18.3 | 34.3 $\pm$ 12.3 |
| <b>Ours</b>          | 12.1 $\pm$ 2.7  | 14.6 $\pm$ 3.0  |
| <i>CLUENER</i>       |                 |                 |
| ProtoBERT            | 39.5 $\pm$ 2.1  | 41.5 $\pm$ 3.3  |
| NNShot               | 4.9 $\pm$ 2.7   | 8.2 $\pm$ 1.3   |
| StructShot           | 39.9 $\pm$ 3.8  | 40.7 $\pm$ 2.6  |
| <b>Ours</b>          | 7.8 $\pm$ 0.9   | 9.1 $\pm$ 0.9   |

Under the same episodic protocol Ours trails ProtoBERT/StructShot by roughly 30 F1 on CoNLL while a Frozen-BERT control collapses near 4 F1; this is an annotation-efficiency stress test of the compact encoder, not a competing few-shot claim.

### E. SEQUENCE-LENGTH SWEEP (GATE G1)

Fixed-length corpora ( $L \in \{16, 32, 64, 96, 128\}$  tokens; target  $n \geq 200$  per length) feed the ESP32 UART harness (esp32-s3-n16r8-espdl-uart-g1; hybrid ESP-DL+firmware blocks for  $L > 73$ ) for latency and stage-share scaling; entity F1 is not re-evaluated on these synthetic corpora.



ESP32-S3 stage latency vs. sentence length (Gate G1). Solid: pure INT8 ( $L \leq 64$ , CoNLL-derived). Dotted: hybrid path ( $L \geq 96$ , uniform tokens); not connected to the INT8 series.

For  $L \in \{16, 32, 64\}$  sentences are CoNLL-trained derived with rotated padding; for  $L \in \{96, 128\}$  uniform test tokens replace real text because long repeated

real tokens stall the on-device embed loop ( $n=184$  and 186 after UART retries). This sweep controls sequence length as the independent variable; synthetic long sequences are not used for entity-F1 evaluation. Figure 6 plots mean stage latency vs.  $L$  (latency columns only in seq\_length\_sweep\_esp32.json). On the pure-INT8 path ( $L \leq 64$ ), embed+quant grows from **1.21 s** to **4.90 s** (about 76 ms/token) while ESP-DL infer stays at **615 ms** ( $\pm 0.3\%$ ). End-to-end latency tracks embed through **1.83 s** ( $L=16$ ) to **5.51 s** ( $L=64$ ). At  $L \in \{96, 128\}$  the hybrid path yields **6.06 s** and **5.55 s** mean e2e with embed at **5.44 s** / **4.88 s** versus infer **618 ms** / **664 ms**. Because  $L > 73$  switches to the hybrid ESP-DL+firmware path and  $L \in \{96, 128\}$  uses uniform-token corpora ( $n=184/186$ ), these two points are not directly comparable with the pure-INT8  $L \leq 64$  series: per-token embed cost falls from about 76 ms to 38 ms across the path switch. Length scaling is interpreted only within each path; the non-monotonic e2e ordering between  $L=96$  and  $L=128$  reflects the path change and corpus composition rather than a reversal of the embed-bound trend. Length scaling on the pure-INT8 path is driven by firmware-side data movement, not by densified conv kernels.

### F. CROSS-LANGUAGE DEPLOYMENT (GATE G2)

Gate G2 reruns the export pipeline on Chinese NER with a 3,673-type character vocabulary (51 types on CoNLL). Checkpoint ckpt-cluener-full-s42 (SHA-256 prefix 0827458f) is exported on CLUENER [33]: INT16 ONNX (1.6 MB) for Pi and a 351 KB INT8 .espdl bundle edgefs-esp32-08549f7c with densified dilations, identity-skip residuals, and Chinese calibration sentences (deploy/shared/cluener\_calib\_samples.json). Section III-E reports the four-tier audit on a 200-sentence prefix before full-board evaluation. On the full CLUENER test split ( $n=1,343$ ), Pi 3B+ and Pi 400 reach **66.9** F1 (P/R 68.1/65.8) at **69.2 ms** and **87.3 ms** mean latency (about 73 MB RSS). ESP32-S3 reaches **66.7** F1 (P/R 68.7/64.9) at **3,473 ms** mean (p50 3,684 ms). F1 drop vs Pi is **0.2** points, well below the 3-point CoNLL gate. Table 9 reports FP32 host training accuracy (67.6); the 0.7 F1 difference from the Gate G2 INT16 deployment (66.9) is post-training quantization and export loss, not a board-evaluation bug. Boot-time espdl\_test checks golden-tensor graph parity only; UART F1 above is the entity-level deploy metric. Stage timers on CLUENER put embed+quant at **82.9%** of MCU time and ESP-DL infer at **17.0%**, a larger embed share than on CoNLL (64%), which matches the larger flash-backed char table. The first CLUENER export omitted densify and identity-skip: espdl\_test failed and on-device F1 fell to about 0 while host PPQ still matched float. The same graph rewrite used for CoNLL was required here. Raw JSON: deploy/results/cluener\_g2\_summary.json and deploy/results/cluener\_p0\_tier\_ladder.json.

Export ablation (Gate G4). A: post-pool + densify + identity-skip (production). B: no identity-skip (Add residual). B': no densify (CLUENER). CoNLL: truncated deploy split ( $n=1,722$ ) except B (smoke20). CLUENER A: full test ( $n=1,343$ ); B/B': smoke20.

| Config         | espd1 test        | CoNLL F1         | CLUENER F1       | Size (KB) | Lat. (ms)         |
|----------------|-------------------|------------------|------------------|-----------|-------------------|
| A: id-skip     | PASS              | 69.5             | 66.7             | 349/351   | 1719/3473         |
| B: no id-skip  | FAIL              | 0.0 <sup>†</sup> | 0.0 <sup>†</sup> | 296       | 2064 <sup>†</sup> |
| B': no densify | FAIL              | —                | 0.0 <sup>†</sup> | 173       | 3483 <sup>†</sup> |
| C: post-embed  | PASS <sup>‡</sup> | —                | —                | 343       | 1228 <sup>‡</sup> |

<sup>†</sup>Smoke20 ( $n=20$ ). F1=0 means the export rewrite is missing and tag output is invalid, not gradual INT8 degradation. <sup>‡</sup>Early self-test ( $n=3$ ). JSON: `ablation_graph_cut.json`.

## G. GRAPH-CUT ABLATION (GATE G4)

Table 11 compares export-time graph rewrites. Config A (post-pool cut, densified dilations, identity  $1\times 1$  skip convolutions) is the path in Tables 7–8: `espd1_test` PASS, CoNLL on-device F1 **69.5** ( $n=1,722$ ), CLUENER F1 **66.7** ( $n=1,343$ ). Config B drops identity-skip and keeps native residual `Add(x)`. Self-test fails and UART F1 goes to **0.0** on both datasets (smoke20), even when host PPQ still matches float on CLUENER, so the residual operator mismatch blocks MCU parity rather than calibration alone. Config B' omits densify on CLUENER: the `.espd1` shrinks to 173 KB but the failure mode is the same. An earlier post-embed prototype (Config C) passed self-test on three English probes at  $\sim 1.23$  s latency but predates the NTC `ConvBlockNTC` layout; we moved to post-pool + identity-skip for full-split F1. Raw JSON: `deploy/results/ablation_graph_cut.json`.

## H. ABLATION: MEMORY VS. OPERATOR LEGALITY

Table 12 isolates C1–C2 on official `eng.testb`. Word-level BiLSTM reaches  $73.4\pm 0.4$  (the same  $k=3$  run as the BiLSTM-CRF row in Table 9; repeated here for the export-budget comparison) but is not MCU-exportable (about 9.1 MB ckpt; OOM/Unsup.). The contrast is flash budget and operator legality under a word-embedding checkpoint, not an encoder-family comparison. Distill Char-CNN is  $71.7\pm 0.8$ ; BERT-teacher distill reaches 72.4 while remaining export-legal. Host PPQ INT8 (69.8) sits within 0.3 of on-device 69.5 (Config A, Table 11).

## I. TRADE-OFF ANALYSIS

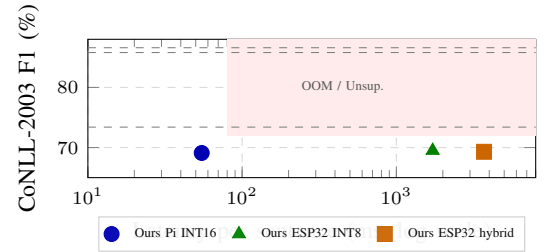
Figure 7 plots measured deploy points (log-scale latency): Pi INT16, ESP32 pure INT8, and optional hybrid. Dashed horizontal guides show `eng.testb` F1 from Table 9 for BiLSTM and transformers; the shaded band marks F1 levels that are infeasible on ESP32-S3 (OOM/Unsup.). The Pi–MCU latency gap (about 54.7 ms vs. about 1.72 s) motivates tiered SKUs under one checkpoint family.

Ablation study (CoNLL-2003). Full / distill / word-level rows use official `eng.testb` ( $n=3,453$ ; 3 seeds where shown). Pi ONNX F1 69.1 and on-device INT8 F1 69.5 use the truncated aligned deploy split ( $n=1,722 / 1,721$ ). “Word-level BiLSTM” repeats the Table 9 BiLSTM-CRF checkpoint for export-budget comparison; it is not MCU-exportable. Best BERT-teacher distill:  $\alpha=0.5$ ,  $T=2$ ,  $\beta=0.1$ .

| Variant                                        | F1            | Size                 | ESP32 Lat.                 | Peak Mem. |
|------------------------------------------------|---------------|----------------------|----------------------------|-----------|
| Full (Ours, 3 seeds @ <code>eng.testb</code> ) | $71.4\pm 0.9$ | 698 KB               | $1,719\text{ ms}^\ddagger$ | 160 KB    |
| Word-level BiLSTM                              | $73.4\pm 0.4$ | $\sim 9.1\text{ MB}$ | Unsup.                     | —         |
| Distill Char-CNN (3 seeds)                     | $71.7\pm 0.8$ | 698 KB               | $1,719\text{ ms}^\ddagger$ | 160 KB    |
| Best distill grid (BERT)                       | 72.4          | 698 KB               | —                          | —         |
| INT8 PPQ host (ESP32 upper bound)              | 69.8          | 349 KB               | —                          | —         |
| On-device INT8 (identity-skip, trunc. test)    | 69.5          | 349 KB               | $1,719\text{ ms}^\ddagger$ | 160 KB    |

<sup>†</sup>Latency shares the verified identity-skip INT8 export; Pi ORT distill 43.5 ms / 58.8 MB (vs. Ours 54.7 ms / 62.2 MB).

<sup>‡</sup>Pure ESP-DL UART on truncated deploy split ( $n=1,721$ ); Table 7.



Deploy accuracy–latency on CoNLL (log latency). Markers: measured Ours deploy F1. Dashed lines: `eng.testb` F1 from Table 9. Shaded band: MCU-infeasible levels (OOM/Unsup.).

## J. DISCUSSION

For latency-first SKUs, Pi fits sub-100 ms targets at about 60 MB RSS. ESP32 fits when BOM and idle power matter more and about 1.7 s per sentence is acceptable (event tagging, buffered logs). Tier choice here follows latency and operator legality, not duty-cycle crossover analysis.

CLUENER char vocab yields about 1 MB embedding tables in firmware; Gate G2 shows the same densify + identity-skip `.espd1` path at 351 KB with 66.7 F1 on MCU and 66.9 on Pi (full test split,  $n=1,343$ ). Byte-level tables are smaller but F1 drops (61.9 in prior sizing experiments); that is a separate accuracy–size trade-off from operator legality.

Unsupported operators block deployment as often as OOM; DistilBERT fails for both. Conv–BN–ReLU with external CRF decode is the pragmatic MCU pattern. Experiments use Espressif ESP-DL on ESP32-S3 as the production runtime with on-device `espd1_test`; TFLite Micro would need a separate operator stack and would not validate the same identity-skip `.espd1` export.

Embed+quant accounts for about 64% of the ESP32 second because variable-length firmware loops walk flash-backed tables per token, while densified ESP-DL kernels are already near length-invariant (about 615 ms). Gate G1 length sweeps quantify this split; firmware-side prefetching and batched token packing are optimizations of the existing graph cut.

Limitations: accuracy trails BiLSTM/transformers by design. Hardware tables report on a truncated official-



testb prefix ( $n \approx 1,722$ ) while main-table accuracy uses full eng.testb (full-testb board re-flash is future work). Pi stage splits are not instrumented. The 128-token MCU cap needs windows for long documents.

EdgeTier-NER is a train-once pipeline for on-device NER under MCU operator constraints. Feasibility tables record OOM and unsupported-operator boundaries for transformer baselines; the Char-CNN-CRF export remains legal on embedded Linux and ESP32-S3. Post-embedding graph cuts, identity-skip residuals, and firmware-side embed/CRF yield a 698 KB Pi ONNX path (54.7 ms, 69.1 F1) and a 349 KB ESP32 .espd1 path with espd1\_test=PASS (69.5 F1 @ 1.72 s). Stage timers show most MCU wall time in firmware embed+quant, which scales with sentence length, while ESP-DL inference stays nearly flat. That split differs from iso-FLOPs vision deployment studies and leaves room for NLP-specific firmware work outside the accelerator graph.

Feasibility tables, memory map, UART protocol, and length-sweep tooling are released with flash/SRAM budgets and runtime legality alongside accuracy. The F1 gap to BiLSTM/BERT on official eng.testb ( $71.4 \pm 0.9$  vs.  $73.4 \pm 0.4$  /  $85.8 \pm 0.4$  /  $86.6 \pm 0.1$ ) is an expected systems trade-off, not the primary claim. CLUENER deployment (Gate G2) reaches 66.9 F1 on Pi INT16 and 66.7 F1 on ESP32 INT8 ( $n=1,343$ ; bundles tagged ckpt-cluener-full-s42 / edgefs-esp32-08549f7c). Raspberry Pi 400 confirms that the same INT16 ONNX reaches 69.1 F1 at 37.6 ms without export changes (Gate G3). Source code, firmware adapters, and evaluation artifacts are publicly available at <https://github.com/T01284/EdgeTier-NER>.

- [1] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. O'Reilly Media, 2019. [Online]. Available: <https://tinymlbook.com/>
- [2] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, "TensorFlow lite micro: Embedded machine learning for TinyML systems," in Proceedings of Machine Learning and Systems (MLSys), vol. 3, 2021. [Online]. Available: <https://proceedings.mlsys.org/paper/2021/73>
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423/>
- [4] Espressif Systems, "ESP-DL: On-device inference framework for ESP32," <https://github.com/espressif/esp-dl>, 2024, accessed 2026-07-12.
- [5] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv preprint arXiv:1704.04861, 2017. [Online]. Available: <https://arxiv.org/abs/1704.04861>
- [6] J. Snell, K. Swersky, and R. S. Zemel, "Prototypical networks for few-shot learning," in Advances in Neural Information Processing Systems, vol. 30, 2017. [Online]. Available: <https://papers.nips.cc/paper/7081-prototypical-networks-for-few-shot-learning>
- [7] Y. Yang and A. Katiyar, "Simple and effective few-shot named entity recognition with structured nearest neighbor learning," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, 2020, pp. 6365–6375. [Online]. Available: <https://aclanthology.org/2020.emnlp-main.516/>
- [8] N. Ding, G. Xu, Y. Chen, X. Wang, X. Han, P. Xie, H. Zheng, and Z. Liu, "Few-NERD: A few-shot named entity recognition dataset," in Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), 2021, pp. 3198–3213. [Online]. Available: <https://aclanthology.org/2021.acl-long.248/>
- [9] J. Huang, C. Li, K. Subudhi, D. Jose, S. Balakrishnan, W. Chen, B. Peng, J. Gao, and J. Han, "Few-shot named entity recognition: An empirical baseline study," in Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, 2021, pp. 10408–10423. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.813/>
- [10] J. Ma, M. Ballesteros, S. Doss, R. Anubhai, S. Mallia, Y. Al-Onaizan, and D. Roth, "Label semantics for few shot named entity recognition," in Findings of the Association for Computational Linguistics: ACL 2022, 2022, pp. 1956–1971. [Online]. Available: <https://aclanthology.org/2022.findings-acl.155/>
- [11] Z. Huang, W. Xu, and K. Yu, "Bidirectional LSTM-CRF models for sequence tagging," arXiv preprint arXiv:1508.01991, 2015. [Online]. Available: <https://arxiv.org/abs/1508.01991>
- [12] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, "Neural architectures for named entity recognition," in Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2016, pp. 260–270. [Online]. Available: <https://aclanthology.org/N16-1030/>
- [13] Espressif Systems, "ESP-DL large language model examples," [https://github.com/espressif/esp-dl/tree/master/examples/large\\_model](https://github.com/espressif/esp-dl/tree/master/examples/large_model), 2024, accessed 2026-09-10.
- [14] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, and P. Kuska, "Natural language processing (almost) from scratch," Journal of Machine Learning Research, vol. 12, pp. 2493–2537, 2011. [Online]. Available: <https://jmlr.org/papers/v12/collobert11a.html>
- [15] X. Ma and E. Hovy, "End-to-end sequence labeling via bi-directional LSTM-CNNs-CRF," in Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2016, pp. 1064–1074. [Online]. Available: <https://aclanthology.org/P16-1101/>
- [16] J. Lafferty, A. McCallum, and F. C. N. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in Proceedings of the Eighteenth International Conference on Machine Learning, 2001, pp. 282–289.
- [17] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in Proceedings of the 34th International Conference on Machine Learning, ser. Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 1126–1135. [Online]. Available: <https://proceedings.mlr.press/v70/finn17a.html>
- [18] O. Vinyals, C. Blundell, T. Lillicrap, K. Kavukcuoglu, and D. Wierstra, "Matching networks for one shot learning," in Advances in Neural Information Processing Systems, vol. 29, 2016, pp. 3630–3638. [Online]. Available: <https://proceedings.neurips.cc/paper/6385-matching-networks-for-one-shot-learning>
- [19] A. Fritzler, V. Logacheva, and M. Kretov, "Few-shot classification in named entity recognition task," in Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing. ACM, 2019, pp. 993–1000. [Online]. Available: <https://dl.acm.org/doi/10.1145/3297280.3297378>
- [20] Y. Hou, W. Che, Y. Lai, Z. Zhou, Y. Liu, H. Liu, and T. Liu, "Few-shot slot tagging with collapsed dependency transfer and label-enhanced task-adaptive projection network," in Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 1381–1393. [Online]. Available: <https://aclanthology.org/2020.acl-main.128/>
- [21] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv preprint arXiv:1503.02531. Presented at NIPS 2015 Deep Learning Workshop, 2015. [Online]. Available: <https://arxiv.org/abs/1503.02531>
- [22] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv preprint arXiv:1910.01108. Presented at NeurIPS 2019 EMC<sup>2</sup> Workshop, 2019. [Online]. Available: <https://arxiv.org/abs/1910.01108>
- [23] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, "TinyBERT: Distilling BERT for natural language

understanding,” in Findings of the Association for Computational Linguistics: EMNLP 2020, 2020, pp. 4163–4174. [Online]. Available: <https://aclanthology.org/2020.findings-emnlp.372/>

- [24] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, “MobileBERT: a compact task-agnostic BERT for resource-limited devices,” in Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 2158–2170. [Online]. Available: <https://aclanthology.org/2020.acl-main.195/>
- [25] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 2704–2713. [Online]. Available: [https://openaccess.thecvf.com/content\\_cvpr\\_2018/html/Jacob\\_Quantization\\_and\\_Training\\_CVPR\\_2018\\_paper.html](https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html)
- [26] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in International Conference on Learning Representations, 2016. [Online]. Available: <https://arxiv.org/abs/1510.00149>
- [27] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, “MCUNet: Tiny deep learning on IoT devices,” in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 11 711–11 722.
- [28] A. Burrello, A. Garofalo, N. Bruschi, G. Tagliavini, D. Rossi, and F. Conti, “DORY: Automatic end-to-end deployment of real-world DNNs on low-cost IoT MCUs,” IEEE Transactions on Computers, vol. 70, no. 8, pp. 1253–1268, 2021.
- [29] A. Choudhury, M. Ghose, A. Islam, and Yogita, “Machine learning-based computation offloading in multi-access edge computing: A survey,” Journal of Systems Architecture, vol. 148, p. 103090, 2024.
- [30] A. Akbik, D. Blythe, and R. Vollgraf, “Contextual string embeddings for sequence labeling,” in Proceedings of the 27th International Conference on Computational Linguistics, 2018, pp. 1638–1649. [Online]. Available: <https://aclanthology.org/C18-1139/>
- [31] Qwen Team, “Qwen3 technical report,” arXiv preprint arXiv:2505.09388, 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [32] S. Hu, C. Tu, X. Han, Z. Huang, Y. Liu et al., “MiniCPM: Unveiling the potential of small language models with scalable training strategies,” arXiv preprint arXiv:2404.06395, 2024. [Online]. Available: <https://arxiv.org/abs/2404.06395>
- [33] L. Xu, T. Yu, Q. Dong, Y. Liao, C. Yu, Y. Tian, W. Liu, L. Li, C. Liu, and X. Zhang, “CLUENER2020: Fine-grained named entity recognition dataset and benchmark for chinese,” arXiv preprint arXiv:2001.04351, 2020. [Online]. Available: <https://arxiv.org/abs/2001.04351>
- [34] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in Proceedings of the 37th International Conference on Machine Learning, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 1597–1607. [Online]. Available: <https://proceedings.mlr.press/v119/chen20j.html>
- [35] E. F. Tjong Kim Sang and F. De Meulder, “Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition,” in Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003, 2003, pp. 142–147. [Online]. Available: <https://aclanthology.org/W03-0419/>
- [36] R. Weischedel, M. Palmer, M. Marcus, E. Hovy, S. Pradhan, L. Ramshaw, N. Xue, A. Taylor, J. Kaufman, M. Franchini, M. El-Bachouti, R. Belvin, and A. Houston, “OntoNotes release 5.0,” Linguistic Data Consortium, LDC2013T19, Philadelphia, PA, 2013. [Online]. Available: <https://catalog.ldc.upenn.edu/LDC2013T19>



MING ZHAO received the LL.M. degree from Northeast Agricultural University, Harbin, China, in 2012. She is currently an Associate Professor with Harbin Cambridge University, Harbin, China, where teaching and research focus on natural language processing and information extraction from domain text. Her research interests include named entity recognition, entity-level evaluation protocol, and structured information extraction from professional documents.



SHU WANG received the B.Sc. degree in computer science from Harbin Institute of Technology, Harbin, China, in 2002. He is currently a Senior Engineer with Heilongjiang Tianyouwei Technology Co., Ltd., Harbin, China. His research interests include embedded Linux and microcontroller inference, operator-feasible model packaging, and industrial IoT software.

...



LI ZHAO received the M.Sc. degree from Harbin Normal University, Harbin, China, in 2009. She is currently an Associate Professor with Heilongjiang Jiaotong Polytechnic, Harbin, China. Her research interests include embedded natural language processing, flash/SRAM-aware encoder budgeting, and reproducible on-board evaluation.